



Namal Criko
Database Design Document
V 3.0

By

Sr.no	Group Member Name	Roll No
1.	M. Jamal Ahmad Khan	NUM-BSCS-2024-51
2.	Qazi M. Auon Farooqi	NUM-BSCS-2024-64
3.	Breera Ijaz	NUM-BSCS-2024-20

Department of Computer Sciences
Namal University
Mianwali, Pakistan

Submission Date: 14th April, 2026

Revision History

Date	Version	Description	Approved by
14/06/26	V 3.0	Added relational database schemas, finalized back-end database implementation constraints, and documented full-stack application implementation.	
13/06/26	V 2.0	Updated version – fixed title page formatting, corrected document structure, and refined section layout.	
15/05/26	V 1.0	Initial database design document – full-stack web application.	

Contents

Revision History	1
1 Project Overview	4
1.1 Introduction	4
1.2 Problem Statement	4
1.3 Project Objectives	5
2 Detailed Database Design	7
2.1 Entity	7
2.2 Data Dictionary	8
2.2.1 User	8
2.2.2 Player	9
2.2.3 Student_Player	9
2.2.4 Other_Player	9
2.2.5 Admin	10
2.2.6 Professional_Player	10
2.2.7 PLAYER_STATISTICS (Batting)	11
2.2.8 PLAYER_STATISTICS (Bowling)	11
2.2.9 Quote	11
2.2.10 QUOTE_IMPRESSION	12
2.2.11 CRICKET_EVENT	12
2.2.12 Tickets	12
2.2.13 Ticket_Response	12
2.2.14 FAQs	13
2.2.15 ADMIN_ACTION_LOG	13
2.2.16 LOGIN_ATTEMPT_LOG	13
2.2.17 PASSWORD_RESET_TOKEN	14
2.2.18 CRICKET_SIMULATION	14
2.2.19 SIMULATION_LOG	14
2.2.20 SPECIFIC_MISTAKE	15
2.2.21 CORRECTION	15
2.2.22 Learning_Streak	15
2.2.23 Asset (Simulation Asset)	15
2.2.24 Category (Simulation Category)	15
2.2.25 Ticket_Category	16
2.2.26 Ticket_History	16
2.3 Business Rules	16
2.4 Entity Relationship Diagram	19
3 Logical Database Design	20
3.1 Relational Schema	20
3.2 Functional Dependencies	22

3.3	Normalization	23
4	System Implementation	25
4.1	Language/Framework	25
4.2	Database Connectivity	26
4.3	Stored Procedures and Functions	27
4.4	System Interface Screenshots	31
	References	34
	Appendix: AI Usage and Prompts	35

Chapter 1: Project Overview

1.1. Introduction

Namal Criko V.01 is a computer-based learning system which has been created for students and people having interest in cricket from Namal University, Mianwali and other Players. The system offers an artificially controlled setting where the users will be able to learn various techniques of playing cricket using visual simulations together with the help of detailed text.

The suggested software solution is designed to function within an educational environment; in particular, university students, interested in cricket but not having chances to practice with the help of a coach and sports ground, will find use for it. The software system has two main categories of users: players and administrators.

The key processes supported by the system include:

- **Cricket Technique Simulations:** Interactive demonstrations of batting and bowling techniques with step-by-step guidance
- **User Management:** Registration, admin approval workflow, profile management, and authentication
- **Performance Tracking:** Automated logging of user activity with progress visualization
- **Help Desk System:** Query submission and resolution tracking
- **Content Management:** Administrative control over simulation content and announcements
- **Top Players Leaderboard:** Motivational display of professional cricket statistics
- **Quote of the Day:** Database-driven motivational quotes for users
- **Events:** Admin can update new events related to sport or can run ads.

The system operates as a standalone web platform accessible via modern web browsers on desktop, tablet, and mobile devices. It is designed to supplement traditional physical training methods by providing 24/7 access to theoretical understanding and visual demonstrations of cricket techniques.

1.2. Problem Statement

Namal University lacks a centralized, technology-driven platform for cricket education and training, resulting in a fragmented and unscalable ecosystem.

On the user side, the absence of persistent progress tracking, performance metrics, and engagement features (such as leaderboards and learning streaks) weaken player focus and training continuity, while the lack of a structured help desk leaves user queries unresolved. Administratively, the system suffers from severe operational overhead due to manual user

lifecycle management, an unstructured simulation catalog for instructional content, and a complete absence of data analytics or automated growth reporting.

Consequently, this manual environment suffers from a degraded user experience, operational inefficiencies, and a lack of data-driven insights to effectively scale cricket training.

1.3. Project Objectives

General Objective:

Develop a production-ready full-stack web application (React + Node.js/Express + MySQL) that serves as a cricket simulation trainer, that supports all features including user management, simulation delivery, performance tracking, help desk support, and administrative reporting.

Specific Objectives:

- Design a normalized relational schema (minimum 3NF) covering all entities including users, simulations, performance logs, help desk tickets, events, top players, and motivational quotes
- Implement all tables, constraints, primary keys, foreign keys, and indexes in MySQL 8.x for data integrity and optimal query performance
- Enable complete user lifecycle management supporting registration, admin approval, profile updates, suspension, and deletion operations
- Store and provide access to the complete simulation catalog (minimum 10 simulations) with sub-second response time to query searches
- Record each player's interaction with the simulation for performance monitoring of up to 100+ simultaneous users
- Control the entire help desk tickets life cycle process while maintaining referential integrity throughout all steps
- Store records for professional cricket players' profiles and achievements, used to generate a motivating "Top Players" list
- Provide a database-driven Quote of the Day feature accessible by each player upon logging in, once every seven days
- Build security and audit logs: login attempts, admin action logs, password reset tokens
- Create a responsive React front-end with:
 - Player dashboard (view simulations, track progress, submit tickets, see Quote of the Day).
 - Admin panel (approve users, manage content, reply to tickets, view reports).
 - Top Players leaderboard with format filters.
 - Quote of the Day display on every login.

GitHub Repository Link:

https://github.com/MJamalAhmadKhan/Namal_Criko_v.2.git

Chapter 2: Detailed Database Design

2.1. Entity

The following entities have been identified as central to the Namal Criko database system. Each entity represents a real-world object, concept, or event that must be tracked within the system.

Sr. No	Entity Name	Description
01	User	All registered users are stored here along with their credentials like email address, password, and account status. This is the starting point for all user authentication and authorization processes.
02	Player	Represents the player in the learning platform of cricket and contains information regarding personal profile of each player such as status, approval info, etc. Has reference to USER.
03	STUDENT_PLAYER	Contains extra information related to academics of student players studying at Namal University. Adds Registration Number & Institute Name attributes to PLAYER entity for identification at the university.
04	OTHER_PLAYER	It stores data about non-student players who use the system. The PLAYER entity is extended to include the name of the organization and the designation/role within the organization.
05	Admin	Represents administrators who manage simulations, tickets, quotes, and users. Linked to USER for authentication.
06	PROFESSIONAL_PLAYER	Stores the details of professional cricketers featured on the leaderboard page. Consists of name, country of origin, position played, style of batting and bowling, biography, and match records.
07	PLAYER STATISTICS	Keeps performance statistics of professional cricket players for various types of games (TEST, ODI, T20). Keeps record of number of matches played, runs scored, batting average, wickets taken, hundreds scored, and last updated date.
08	QUOTE	Displays motivational quotes that relate to the sport of cricket to the user. Includes the quote itself, the name of the author, category (BATTING/BOWLING/GENERAL/MINDSET), whether it is active, and timestamp data.

Sr. No	Entity Name	Description
09	QUOTE_IMPRESSION	Tracks which quote was shown to which player and when. Enforces the 7-day non-repeat rule by logging show date for each quote per player.
10	EVENT	Stores cricket-related events, announcements, and activities. Includes event title, date, venue, description, registration link, and archive status.
11	TICKETS	Stores help desk queries submitted by players. Contains ticket subject, category, description, status (OPEN/IN_PROGRESS/RESOLVED/CLOSED), and timestamps.
12	TICKET_RESPONSE	Stores admin responses against submitted help desk tickets. Links to TICKETS and ADMIN entities, contains response description, attachment, and timestamp.
13	FAQS	Stores frequently asked questions and their answers. Helps users find quick solutions to common queries without submitting tickets.
14	ADMIN_ACTION_LOG	Maintains logs of activities performed by administrators. Tracks action type and timestamp for compliance and auditing.
15	LOGIN_ATTEMPT_LOG	Stores login history and authentication attempts for security purposes. Records IP address, success/failure status, and timestamp for detecting suspicious activities.
16	PASSWORD_RESET_TOKEN	Stores password recovery tokens and expiration details. Contains hashed token, expiration timestamp, and usage status for secure password reset functionality.
17	CRICKET_SIMULATION	Stores cricket learning simulations including batting and bowling techniques. Contains simulation name, duration, and deletion tracking.
18	SIMULATION_LOG	Tracks simulation usage, completion status, and player activity. Records which player accessed which simulation, view time, and completion flag.
19	SPECIFIC_MISTAKE	Stores common mistakes associated with cricket techniques and simulations. Each mistake is linked to a specific simulation and includes mistake description.
20	CORRECTION	Stores correction tips and improvement guidelines for specific mistakes. Links to SPECIFIC_MISTAKE.
21	Learning Streak	Stores data like last login, current streak and max streak.
22	Asset (Simulation Asset)	Store assets like URLs of video material.
23	Category (Simulation Category)	Store categories of simulation.
24	Ticket_Category	Has Ticket category for organizing issues.
25	Ticket_History	Is used to store ticket record changes.

2.2. Data Dictionary

2.2.1. User

Sr.No	Name	Data Type	Constraint	Description
1	user_id	INT	PK, AUTO-INC, NOT NULL	Unique identifier for each user
2	first_name	VARCHAR(50)	NOT NULL	First name of the user
3	last_name	VARCHAR(50)	NOT NULL	Last name of the user
4	email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for login
5	phone_number	VARCHAR(20)	NULL	Contact phone number
6	password_hash	VARCHAR(255)	NOT NULL	Bcrypt encrypted password
7	Created_at	DATE	NOT NULL, DEFAULT NOW()	Account creation timestamp
8	Deactivated_at	DATE	NULL	Timestamp when account deactivated
9	is_Deleted	BOOLEAN	DEFAULT FALSE	Soft delete flag

2.2.2. Player

Sr.No	Name	Data Type	Constraint	Description
1	player_id	INT	PK, FK, NOT NULL	Unique identifier for the player
2	status	ENUM	NOT NULL	Current status of the player account
3	is_Student	BOOLEAN	NOT NULL, DEFAULT FALSE	Flag: player is a student
4	is_Other	BOOLEAN	NOT NULL, DEFAULT FALSE	Flag: player is a non-student
5	Approved_BY	INT	FK, NULL	Reference to ADMIN who approved
6	Approve_Date	DATETIME	NULL	Timestamp when player was approved

2.2.3. Student_Player

Sr.No	Name	Data Type	Constraint	Description
1	player_id	INT	PK, FK, NOT NULL	Reference to PLAYER table
2	InstituteID	INT	FK, NOT NULL	Reference to Institute
3	registration_number	VARCHAR(50)	UNIQUE, NOT NULL	University registration number

2.2.4. Other_Player

Sr.No	Name	Data Type	Constraint	Description
1	player_id	INT	PK, FK, NOT NULL	Reference to PLAYER table
2	OrgID	INT	FK, NOT NULL	Reference to Organization table
3	designation	VARCHAR(100)	NOT NULL	Role within the organization

Organization

Sr.No	Name	Data Type	Constraint	Description
1	OrgID	INT	PK, AUTO.INC, NOT NULL	Unique identifier for organization
2	Name	VARCHAR(100)	NOT NULL	Name of the organization
3	Province	VARCHAR(50)	NULL	Province of the organization
4	City	VARCHAR(50)	NULL	City of the organization
5	Country	VARCHAR(50)	NULL	Country of the organization

2.2.5. Admin

Sr.No	Name	Data Type	Constraint	Description
1	admin_id	INT	PK,FK NOT NULL	Unique identifier for admin record

2.2.6. Professional Player

Sr.No	Name	Data Type	Constraint	Description
1	player_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for professional player
2	Name	VARCHAR(100)	NOT NULL	Full name of the professional cricketer
3	Nationality	VARCHAR(50)	NOT NULL	Country the player represents
4	role	ENUM	NOT NULL	Primary role (Batsman/Bowler/All-Rounder)
5	style	ENUM	NULL	Playing style
6	Created_AT	DATETIME	DEFAULT NOW()	Timestamp when player was added
7	Biography	TEXT	NULL	Brief career summary
8	Deleted_at	DATETIME	NULL	Soft delete timestamp
9	is_Deleted	BOOLEAN	DEFAULT FALSE	Soft delete flag
10	Matches_Played	INT	DEFAULT 0	Total matches played

2.2.7. PLAYER_STATISTICS (Batting)

Base Table: **PLAYER_STATISTICS**

Sr.No	Name	Data Type	Constraint	Description
1	stat_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for statistic record
2	player_id	INT	FK, NOT NULL	Reference to PROFESSIONAL_PLAYER
3	format	ENUM	NOT NULL	Cricket format (TEST/ODI/T20/PSL/IPL/Other)
4	Last_Update	DATETIME	NOT NULL	Date of last statistic update
5	Matches_Played	INT	DEFAULT 0	Number of matches played

Sub-Table: **Batting_STATISTICS**

Sr.No	Name	Data Type	Constraint	Description
1	stat_id	INT	PK, FK, NOT NULL	Reference to Base PLAYER_STATISTICS
2	runs_scored	INT	DEFAULT 0	Total runs scored
3	fifties	INT	DEFAULT 0	Number of 50–99 scores
4	Centuries	INT	DEFAULT 0	Number of 100+ scores

2.2.8. PLAYER_STATISTICS (Bowling)

Sr.No	Name	Data Type	Constraint	Description
1	stat_id	INT	PK, FK, NOT NULL	Reference to Base PLAYER_STATISTICS
2	average_Wicket_Taken	FLOAT	DEFAULT 0.00	Bowling average per wicket
3	wickets_taken	INT	DEFAULT 0	Total wickets taken (integer)

2.2.9. Quote

Sr.No	Name	Data Type	Constraint	Description
1	quote_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for the quote
2	quote_text	VARCHAR	UNIQUE, NOT NULL	The motivational quote content
3	category	ENUM	NOT NULL	Theme category of the quote
4	active_flag	BOOLEAN	DEFAULT TRUE	Whether quote is available for display
5	Author	VARCHAR(100)	NOT NULL	Person who said the quote
6	Created_at	DATETIME	DEFAULT NOW()	Timestamp when quote was added

2.2.10. QUOTE_IMPRESSION

Sr.No	Name	Data Type	Constraint	Description
1	impression_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for impression
2	player_id	INT	FK, NOT NULL	Reference to the player
3	quote_id	INT	FK, NOT NULL	Reference to the quote shown
4	shown_date	DATETIME	NOT NULL	Date and time quote was shown

2.2.11. CRICKET_EVENT

Sr.No	Name	Data Type	Constraint	Description
1	event_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the event
2	event_name	VARCHAR(200)	NOT NULL	Name/title of the event
3	event_date	DATETIME	NOT NULL	Date and time of the event
4	venue	VARCHAR(200)	NOT NULL	Location of the event
5	archived_flag	BOOLEAN	DEFAULT FALSE	Whether event has been archived
6	Image_Url	VARCHAR(500)	NULL	URL for event display image
7	Created_By_Admin	INT	FK	Reference to ADMIN who created event
8	Description	TEXT	NOT NULL	Detailed information about the event
9	Created_AT	DATETIME	DEFAULT NOW()	Timestamp when event was posted
10	Registration_Link	VARCHAR(500)	NULL	URL for event registration

2.2.12. Tickets

Sr.No	Name	Data Type	Constraint	Description
1	ticket_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the ticket
2	player_id	INT	FK, NOT NULL	Reference to the player
3	subject	VARCHAR(200)	NOT NULL	Brief summary of the issue
4	status	ENUM	DEFAULT 'Open'	Current status of the ticket
5	Category_Id	INT	FK, NOT NULL	Reference to Ticket Category
6	Attachment_URL	VARCHAR(500)	NULL	Path to any attached file
7	Submission_Date	DATETIME	DEFAULT NOW()	Submission timestamp
8	Description	TEXT	NOT NULL	Detailed explanation of the issue

2.2.13. Ticket_Response

Sr.No	Name	Data Type	Constraint	Description
1	response_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the response
2	ticket_id	INT	Flea, NOT NULL	Reference to the ticket
3	admin_id	INT	FK, NOT NULL	Reference to the ADMIN who responded
4	description	TEXT	NOT NULL	Content of the response
5	is_successful	BOOLEAN	DEFAULT TRUE	Whether response resolved the issue
6	Attachment_URI	VARCHAR(500)	NULL	Path to attachment in the response
7	Response_Time	DATETIME	DEFAULT NOW()	Timestamp when response was posted

2.2.14. FAQs

Sr.No	Name	Data Type	Constraint	Description
1	faq_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the FAQ
2	question	TEXT	NOT NULL	The frequently asked question
3	answer	TEXT	NOT NULL	The answer to the question
4	admin_id	INT	FK	Reference to ADMIN who created FAQ

2.2.15. ADMIN_ACTION_LOG

Sr.No	Name	Data Type	Constraint	Description
1	log_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for audit record
2	admin_id	INT	FK, NOT NULL	Reference to ADMIN who performed action
3	action_type	VARCHAR(50)	NOT NULL	Type of action performed
4	description	TEXT	NOT NULL	Description of the action
5	performed_at	DATETIME	DEFAULT NOW()	Timestamp when action was performed

2.2.16. LOGIN_ATTEMPT_LOG

Sr.No	Name	Data Type	Constraint	Description
1	log_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for login attempt
2	UserId	INT	FK, NOT NULL	Reference to USER attempting login
3	ip_address	VARCHAR(45)	NOT NULL	IP address of the attempt
4	is_successful	BOOLEAN	NOT NULL	Whether login succeeded

Sr.No	Name	Data Type	Constraint	Description
5	attempted_at	DATETIME	DEFAULT NOW()	Timestamp of login attempt

2.2.17. PASSWORD_RESET_TOKEN

Sr.No	Name	Data Type	Constraint	Description
1	token_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for reset token
2	User_id	INT	FK, NOT NULL	Reference to USER requesting reset
3	token_hash	VARCHAR(255)	NOT NULL	Hashed reset token for security
4	expires_at	DATETIME	NOT NULL	Timestamp token becomes invalid
5	is_used	BOOLEAN	DEFAULT FALSE	Whether token has been used

2.2.18. CRICKET_SIMULATION

Sr.No	Name	Data Type	Constraint	Description
1	simulation_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the simulation
2	simulation_name	VARCHAR(100)	UNIQUE, NOT NULL	Name of the simulation
3	duration	INT	DEFAULT 120	Estimated learning duration
4	Deleted_at	DATE	NULL	Soft delete timestamp
5	is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete flag
6	CategoryID	INT	FK, NOT NULL	Reference to Simulation Category

2.2.19. SIMULATION_LOG

Sr.No	Name	Data Type	Constraint	Description
1	log_id	INT	PK, AUTO.INC, NOT NULL	Unique identifier for the log record
2	player_id	INT	FK, NOT NULL	Reference to PLAYER who accessed simulation
3	view_time	DATETIME	NOT NULL, DEFAULT NOW()	Date and time of access
4	is_completed	BOOLEAN	DEFAULT FALSE	Whether player completed the simulation
5	Time.Spend_sec	INT	NULL	Total time spent on simulation in seconds

Sr.No	Name	Data Type	Constraint	Description
6	Simulation_Id	INT	FK, NOT NULL	Reference to the simulation accessed

2.2.20. SPECIFIC_MISTAKE

Sr.No	Name	Data Type	Constraint	Description
1	simulation_id	INT	PK, FK, NOT NULL	Reference to the simulation
2	mistake_desc	VARCHAR	PK, NOT NULL	Description of the common mistake

2.2.21. CORRECTION

Sr.No	Name	Data Type	Constraint	Description
1	correction_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for the correction
2	mistake_id	INT	FK, NOT NULL	Reference to the mistake being corrected

2.2.22. Learning_Streak

Sr.No	Name	Data Type	Constraint	Description
1	LearningStreak_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for the streak record
2	Player_Id	INT	FK, NOT NULL	Reference to the PLAYER
3	Last_Login	DATETIME	NULL	Date and time of last login
4	Current_Streak	INT	DEFAULT 0	Current consecutive login streak
5	Max_Streak	INT	DEFAULT 0	Maximum streak ever achieved

2.2.23. Asset (Simulation Asset)

Sr.No	Name	Data Type	Constraint	Description
1	Asset_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for the asset
2	Simulation_id	INT	FK, NOT NULL	Reference to CRICKET_SIMULATION
3	Asset_Type	ENUM	NOT NULL	Type of the asset (Video/Image/Text)
4	Asset_URI	VARCHAR(500)	NOT NULL	URL/path to the asset

2.2.24. Category (Simulation Category)

Sr.No	Name	Data Type	Constraint	Description
1	Category_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for the category
2	Parent_Category	INT	FK, NULL	Reference to parent category
3	Name	VARCHAR(100)	NOT NULL	Name of the category
4	Difficulty_Level	INT	NULL	Difficulty level for this category

2.2.25. Ticket_Category

Sr.No	Name	Data Type	Constraint	Description
1	Cat_Id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for ticket category
2	Name	VARCHAR(100)	NOT NULL	Name of the ticket category
3	Priority_Level	INT	NOT NULL	Priority level for this category

2.2.26. Ticket_History

Sr.No	Name	Data Type	Constraint	Description
1	ticket_h_id	INT	PK, AUTO_INC, NOT NULL	Unique identifier for ticket history
2	ticket_id	INT	FK, NOT NULL	Reference to the TICKETS table
3	Old_Status	ENUM	NOT NULL	Previous status of the ticket
4	New_Status	ENUM	NOT NULL	New status of the ticket
5	Changed_By	INT	FK, NOT NULL	User/admin who changed the status
6	Update_Date	DATETIME	DEFAULT NOW()	Timestamp when status was changed

2.3. Business Rules

User Management Rules

- Every user must have a unique email address
- Password must be stored as bcrypt hash, never as plain text
- New user accounts have status 'PENDING' until approved by admin
- Only users with role 'ADMIN' can access admin panel
- Only 'ACTIVE' players can access simulation content
- A player can be EITHER a STUDENT_PLAYER OR an OTHER_PLAYER, not both
- Registration number must be unique for student players

- Soft delete implemented via deleted_at timestamp

Simulation Rules

- Simulation name must be unique across all simulations
- Each simulation has type: BATTING, BOWLING, or FIELDING
- Difficulty level is mandatory: BEGINNER, INTERMEDIATE, or ADVANCED
- View count auto-increments when simulation is accessed
- Simulations can be soft-deleted (deleted_at)

Mistake and Correction Rules

- Each SPECIFIC_MISTAKE belongs to exactly one CRICKET_SIMULATION
- Each CORRECTION belongs to exactly one SPECIFIC_MISTAKE
- A simulation can have multiple mistakes
- A mistake can have exactly one correction (one-to-one relationship)
- Severity level indicates how critical the mistake is for performance

Performance Tracking Rules

- Every simulation access creates a SIMULATION_LOG record
- Duration seconds cannot be negative
- is_completed = TRUE only when user views entire simulation
- Session groups multiple accesses within a single user session

Help Desk Rules

- Only players can create tickets (not admins)
- Ticket status flow: OPEN → IN_PROGRESS → RESOLVED → CLOSED
- Only admins can update ticket status
- Only admins can add responses to tickets
- Closed tickets cannot be reopened

FAQ Rules

- FAQs can be categorized for easy navigation
- Display order determines the order of appearance
- is_active = FALSE hides FAQ from users

Quote Rules

- Same quote cannot be shown to same player within 7 days
- is_active = FALSE quotes are not displayed
- Only admins can add, edit, or delete quotes

- Quote impressions are logged automatically when displayed

Event Rules

- Events with past event date should be auto-archived
- Only admins can create, edit, or delete events
- Deleted events are soft-deleted

Password Reset Rules

- Tokens expire after 1 hour (expires_at)
- Used tokens cannot be reused (is_used = TRUE)
- One user can have multiple tokens but only latest valid one should be used

Security Rules

- All login attempts logged regardless of success
- Failed attempts store failure reason
- All admin actions logged in ADMIN_ACTION_LOG
- Admin action log stores old and new values as JSON

Professional Player Rules

- A professional player can have statistics for multiple formats
- Statistics must be updated regularly
- Leaderboard ranks by runs (batters) or wickets (bowlers)

2.4. Entity Relationship Diagram

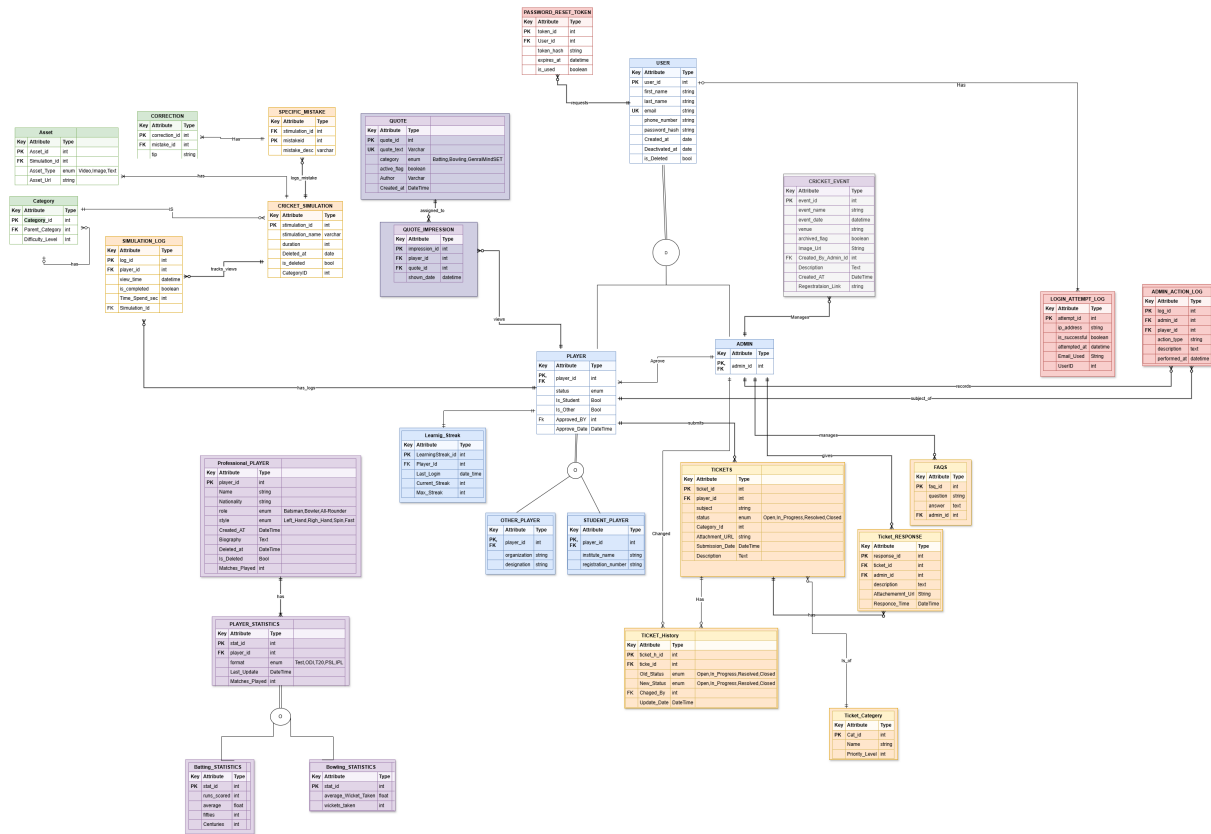


Figure 2.1: Entity-Relationship Diagram (ERD)

Chapter 3: Logical Database Design

3.1. Relational Schema

The conceptual Entity-Relationship Diagram (ERD) is translated into a standardized physical relational schema execution model. Attributes mapping to Primary Keys (PK) are marked explicitly, and contextual pointers linking distinct data domains together map using Foreign Keys (FK).

1. **USER** (user_id, first_name, last_name, email, phone_number, password_hash, Created_at, Deactivated_at, is_Deleted)
2. **ADMIN** (admin_id, *UserId*)
Foreign Key: UserId references USER(user_id)
3. **PLAYER** (player_id, status, Is_Student, Is_Other, *Approved_BY*, Approve_Date)
Foreign Key: Approved_BY references ADMIN(admin_id)
4. **PROFESSIONAL_PLAYER** (player_id, Name, Nationality, role, style, Created_AT, Biography, Deleted_at, Is_Deleted, Matches_Played)
Foreign Key: player_id references PLAYER(player_id)
5. **ORGANIZATION** (ORGID, Name, Province, City, Country)
6. **OTHER_PLAYER** (player_id, *OrgID*, designation)
Foreign Key: player_id references PLAYER(player_id)
Foreign Key: OrgID references ORGANIZATION(ORGID)
7. **INSTITUTE** (InstituteID, Name, Province, City, Country)
8. **STUDENT_PLAYER** (player_id, *InstituteID*, registration_number)
Foreign Key: player_id references PLAYER(player_id)
Foreign Key: InstituteID references INSTITUTE(InstituteID)
9. **PLAYER_STATISTICS** (stat_id, *player_id*, format, Last_Update, Matches_Played)
Foreign Key: player_id references PROFESSIONAL_PLAYER(player_id)
10. **BATTING_STATISTICS** (stat_id, runs_scored, average, fifties, Centuries)
Foreign Key: stat_id references PLAYER_STATISTICS(stat_id)
11. **BOWLING_STATISTICS** (stat_id, wickets_taken, average_Wicket_Taken)
Foreign Key: stat_id references PLAYER_STATISTICS(stat_id)
12. **CATEGORY** (Category_id, *Parent_Category*, Name, Difficulty_Level)
Foreign Key: Parent_Category references CATEGORY(Category_id)
13. **CRICKET_SIMULATION** (simulation_id, simulation_name, duration, deleted_at, is_deleted, *categoryID*)
Foreign Key: categoryID references CATEGORY(Category_id)
14. **SIMULATION_LOG** (log_id, *player_id*, view_time, is_completed, Time_Spend_sec, *Simulation_Id*)

- Foreign Key:* `player_id` references `PLAYER(player_id)`
Foreign Key: `Simulation_Id` references `CRICKET_SIMULATION(simulation_id)`
15. **SPECIFIC_MISTAKE** (`mistake_id`, `stimulation_id`, `mistake_desc`)
Foreign Key: `stimulation_id` references `CRICKET_SIMULATION(simulation_id)`
 16. **CORRECTION** (`correction_id`, `mistake_id`, `tip`)
Foreign Key: `mistake_id` references `SPECIFIC_MISTAKE(mistake_id)`
 17. **QUOTE** (`quote_id`, `quote_text`, `category`, `active_flag`, `Author`, `Created_at`)
 18. **QUOTE_IMPRESSION** (`impression_id`, `player_id`, `quote_id`, `shown_date`)
Foreign Key: `player_id` references `PLAYER(player_id)`
Foreign Key: `quote_id` references `QUOTE(quote_id)`
 19. **CRICKET_EVENT** (`event_id`, `event_name`, `event_date`, `venue`, `archived_flag`, `Image_URL`, `Created_By_Admin_Id`, `Description`, `Created_AT`, `Registration_Link`)
Foreign Key: `Created_By_Admin_Id` references `ADMIN(admin_id)`
 20. **TICKET_CATEGORY** (`Cat_id`, `Name`, `Priority_Level`)
 21. **TICKETS** (`ticket_id`, `player_id`, `subject`, `status`, `Category_Id`, `Attachment_URL`, `Submission_Date`, `Description`)
Foreign Key: `player_id` references `PLAYER(player_id)`
Foreign Key: `Category_Id` references `TICKET_CATEGORY(Cat_id)`
 22. **TICKET_HISTORY** (`ticket_h_id`, `ticket_id`, `Old_Status`, `New_Status`, `Changed_By`, `Update_Date`)
Foreign Key: `ticket_id` references `TICKETS(ticket_id)`
Foreign Key: `Changed_By` references `USER(user_id)`
 23. **TICKET_RESPONSE** (`response_id`, `ticket_id`, `admin_id`, `description`, `Attachment_Url`, `Response_Time`)
Foreign Key: `ticket_id` references `TICKETS(ticket_id)`
Foreign Key: `admin_id` references `ADMIN(admin_id)`
 24. **FAQS** (`faq_id`, `question`, `answer`, `admin_id`)
Foreign Key: `admin_id` references `ADMIN(admin_id)`
 25. **PASSWORD_RESET_TOKEN** (`token_id`, `User_id`, `token_hash`, `expires_at`, `is_used`)
Foreign Key: `User_id` references `USER(user_id)`
 26. **LOGIN_ATTEMPT_LOG** (`attempt_id`, `User_id`, `ip_address`, `is_successful`, `attempted_at`, `Email`)
Foreign Key: `User_id` references `USER(user_id)`
 27. **ADMIN_ACTION_LOG** (`log_id`, `admin_id`, `player_id`, `action_type`, `description`, `performed_at`)
Foreign Key: `admin_id` references `ADMIN(admin_id)`
Foreign Key: `player_id` references `PLAYER(player_id)`
 28. **LEARNING_STREAK** (`LearningStreak_id`, `Player_id`, `Last_Login`, `Current_Streak`, `Max_Streak`)
Foreign Key: `Player_id` references `PLAYER(player_id)`

3.2. Functional Dependencies

Functional dependencies ($X \rightarrow Y$) are systematically verified for every relation to guarantee relational integrity and mathematical completeness.

- **USER:**

$user_id \rightarrow \{first_name, last_name, email, phone_number, password_hash, Created_at, Deactivated_at, email \rightarrow user_id$

- **ADMIN:** $admin_id \rightarrow \emptyset$

- **PLAYER:** $player_id \rightarrow \{status, Is_Student, Is_Other, Approved_BY, Approve_Date\}$

- **PROFESSIONAL_PLAYER:** $player_id \rightarrow \{Name, Nationality, role, style, Created_AT, Biography\}$

- **ORGANIZATION:** $ORGID \rightarrow \{Name, Province, City, Country\}$

- **OTHER_PLAYER:** $player_id \rightarrow \{OrgID, designation\}$

- **INSTITUTE:** $InstituteID \rightarrow \{Name, Province, City, Country\}$

- **STUDENT_PLAYER:** $player_id \rightarrow \{InstituteID, registration_number\}$

- **PLAYER_STATISTICS:** $stat_id \rightarrow \{player_id, format, Last_Update, Matches_Played\}$

- **BATTING_STATISTICS:** $stat_id \rightarrow \{runs_scored, average, fifties, Centuries\}$

- **BOWLING_STATISTICS:** $stat_id \rightarrow \{wickets_taken, average_Wicket_Taken\}$

- **CATEGORY:** $Category_id \rightarrow \{Parent_Category, Name, Difficulty_Level\}$

- **CRICKET_SIMULATION:** $simulation_id \rightarrow \{simulation_name, duration, deleted_at, is_deleted, c\}$

- **SIMULATION_LOG:** $log_id \rightarrow \{player_id, view_time, is_completed, Time_Spend_sec, Simulation_I\}$

- **SPECIFIC_MISTAKE:** $mistake_id \rightarrow \{stimulation_id, mistake_desc\}$

- **CORRECTION:** $correction_id \rightarrow \{mistake_id, tip\}$

- **QUOTE:** $quote_id \rightarrow \{quote_text, category, active_flag, Author, Created_at\}$

- **QUOTE_IMPRESSION:** $impression_id \rightarrow \{player_id, quote_id, shown_date\}$

- **CRICKET_EVENT:** $event_id \rightarrow \{event_name, event_date, venue, archived_flag, Image_URL, Creat\}$

- **TICKET_CATEGORY:** $Cat_id \rightarrow \{Name, Priority_Level\}$

- **TICKETS:** $ticket_id \rightarrow \{player_id, subject, status, Category_Id, Attachment_URL, Submission_Dat\}$

- **TICKET_HISTORY:** $ticket_h_id \rightarrow \{ticket_id, Old_Status, New_Status, Changed_By, Update_Date\}$

- **TICKET_RESPONSE:** $response_id \rightarrow \{ticket_id, admin_id, description, Attachement_Url, Respon\}$

- **FAQS:** $faq_id \rightarrow \{question, answer, admin_id\}$

- **PASSWORD_RESET_TOKEN:** $token_id \rightarrow \{User_id, token_hash, expires_at, is_used\}$

- **LOGIN_ATTEMPT_LOG:** $attempt_id \rightarrow \{User_id, ip_address, is_successful, attempted_at, Email\}$

- **ADMIN_ACTION_LOG:** $log_id \rightarrow \{admin_id, player_id, action_type, description, performed_at\}$

- **LEARNING_STREAK:** LearningStreak_id $\rightarrow$ {Player_id, Last_Login, Current_Streak, Max_Streak}

3.3. Normalization

Structural design patterns are step-analyzed to verify conformance to First, Second, and Third Normal Forms (1NF, 2NF, 3NF). Normalization steps address data redundancies and anomaly potentials within structural fields.

Applied Structural Optimizations

a. Normalization Change #1: QUOTE_IMPRESSION Table Optimization

- *Before:* QUOTE_IMPRESSION (impression_id, player_id, quote_id, shown_date, Days_Since_Repeated)
- *After:* QUOTE_IMPRESSION (impression_id, player_id, quote_id, shown_date)
- *Reasoning:* The attribute Days_Since_Repeated represents a derived metric calculated directly from computing the variance against shown_date. Removing this prevents update and computational tracking anomalies.

b. Normalization Change #2: Self-Referential CATEGORY Refinement

- *Before:* CATEGORY (Category_id, Parent_Category, Sub_Category, level)
- *After:* CATEGORY (Category_id, Parent_Category, Name, Difficulty_Level)
- *Reasoning:* The inclusion of an explicit Sub_Category field introduced dependency duplication. Tree hierarchies are completely represented via a clean self-referential single foreign relationship pointer (Parent_Category).

c. Normalization Change #3: USER Temporal Standardization

- *Before:* USER (user_id, ..., start_date, end_date, ...)
- *After:* USER (user_id, ..., Created_at, Deactivated_at, ...)
- *Reasoning:* Field descriptions were renamed to conform with structured audit tracking and consistency standards.

Verification of Normal Forms

- **1NF Verification:** Atomic attributes are maintained across all records. Multi-valued fields and nested array listings do not exist. Repeating arrays are split completely into structural sub-logs (e.g., TICKET_HISTORY, SIMULATION_LOG).
- **2NF Verification:** The relations are in 1NF, and every non-prime field demonstrates full functional dependency on the complete primary key structure. There are no partial key dependencies.
- **3NF Verification:** All relations conform to 2NF requirements. Transitive dependencies are structurally eliminated ($X \rightarrow Y \rightarrow Z$ does not occur). Non-key operational metrics (like organizational specifics or institutional locations) are segmented entirely into their own base entities (ORGANIZATION, INSTITUTE) to isolate modification footprints.

The structural schema successfully maps as a clean, normalized, 3NF-compliant physical configuration model.

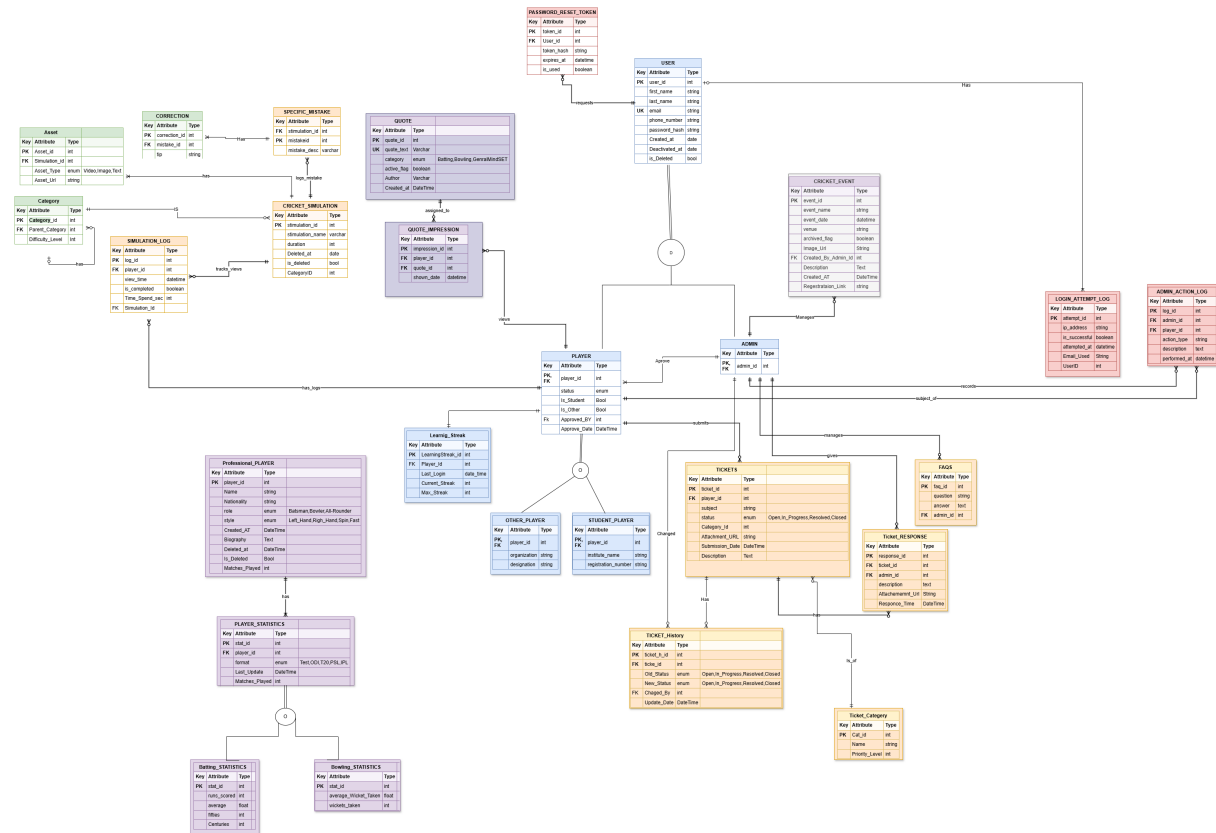


Figure 3.1: Normalized Relational Schema

Chapter 4: System Implementation

4.1. Language/Framework

The system is developed using a robust, full-stack JavaScript architecture supported by a relational database management system. The specific technologies, programming languages, and frameworks used, along with their selection suitability, are outlined below:

A. GUI Client (Frontend)

- **Framework/Library:** React (TypeScript)
- **Build Tool:** Vite
- **Rationale & Advantages:**
 - React’s component-based design allows the user interface to be divided into modular, reusable blocks (such as the `SimulationViewer`, `AdminPanel`, and `HelpDesk` components), ensuring a highly maintainable and clean front-end.
 - TypeScript provides static type definitions (e.g., interface structures for `TopPlayer`, `FAQ`, `CricketEvent`, etc.), catching syntax and type mismatches at compile-time rather than during client execution.
 - Vite enables extremely rapid Hot Module Replacement (HMR) and uses ES modules during development, significantly boosting developer productivity and ensuring highly optimized distribution files.

B. Application Server (Backend)

- **Language:** Node.js (JavaScript)
- **Framework:** Express
- **Rationale & Advantages:**
 - Full-stack JavaScript consistency simplifies developers’ workflow by using the same programming language for both client-side and server-side logic.
 - Express provides lightweight, flexible routing capabilities, middleware support (e.g., JWT authentication checks, activity logging), and fast JSON API response handling.
 - Node’s asynchronous, non-blocking I/O model is ideal for real-time applications, database queries, and concurrent user management.

C. Relational Database

- **Rationale & Advantages:** MySQL
 - Relational schema structure enforces data integrity rules and relational links between players, organizations, support tickets, and logs.

- Robust support for Database Programmability (Stored Procedures, Triggers, Views, and Functions) allows business and authorization logic to be compiled and executed directly inside the database engine, minimizing data transfer and securing data integrity.

4.2. Database Connectivity

A. Connectivity Library & Driver

The backend server communicates with the MySQL server using the `mysql2/promise` npm library. This is a modern, promise-wrapped wrapper for the classic MySQL client that supports:

- Promise-based asynchronous execution (allowing clean `async/await` syntax).
- Prepared statements, preventing SQL injection vulnerabilities.
- Connection Pooling, which manages a cache of active database connections, eliminating the overhead of opening and closing connections on every API request.

B. Connection Parameters

The connection configuration is managed in “backend/db.js” and is dynamically loaded from a local environment file (`.env`) to protect sensitive credentials:

- **DB_HOST:** “localhost” (The location of the MySQL server)
- **DB_USER:** “root” (Database access user)
- **DB_PASSWORD:** “Jamal543@” (Database authentication password)
- **DB_NAME:** “namalcriko” (Relational database schema name)

C. Communication Architecture

The project implements a 3-Tier Architecture for database communication:

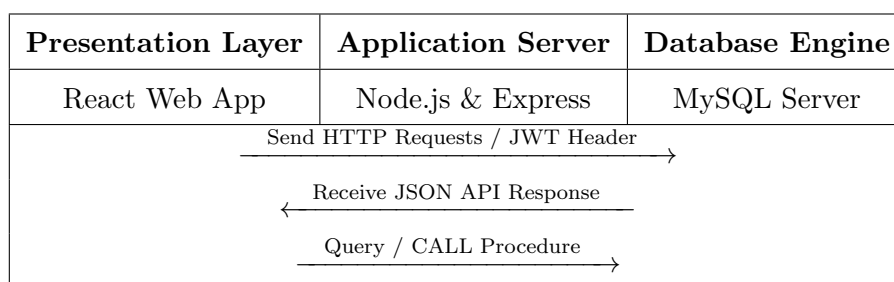


Figure 4.1: 3-Tier Communication Architecture

1. **Presentation Layer:** The React frontend initiates API requests (e.g., player logins, simulation requests, ticket submissions) using Axios or Fetch to the backend port 5000.
2. **Application Layer:** The Express server intercepts the API request, validates the client’s authorization token (JWT), retrieves connection instances from the pool, and runs query executions.

3. **Database Layer:** The MySQL server parses the queries, runs transaction locks, calls stored procedures, triggers audits, and returns data tuples to the server.

4.3. Stored Procedures and Functions

Below is a complete index of all database programmability objects developed for the project, detailing their object type, purpose, parameters, and output results:

Object Name	Type	Description	Input Parameters	Output / Return Value
fn_get_player_status	Function	Gets current status of player.	p_player_id INT	VARCHAR(20) (Status value)
fn_is_token_valid	Function	Validates password reset token expiration and status.	p_token_hash VARCHAR(255)	BOOLEAN (True if active/valid)
fn_player_simulations_completed	Function	Counts how many simulations a player has successfully completed.	p_player_id INT	INT (Count of completed logs)
fn_open_tickets_count	Function	Counts active unresolved support tickets submitted by a player.	p_player_id INT	INT (Count of open tickets)
sp_register_user	Procedure	Registers new user, player, and streak record in database.	p_first_name VARCHAR(100), p_last_name VARCHAR(100), p_email VARCHAR(150), p_phone VARCHAR(50), p_password_hash VARCHAR(255), p_user_type ENUM('Student', 'Other'), p_org_id INT, p_reg_number VARCHAR(50), p_designation VARCHAR(100)	p_new_user_id INT (New ID), p_message VARCHAR(200) (Status)

Object Name	Type	Description	Input Parameters	Output / Return Value
sp_login_attempt	Procedure	Logs access attempt, verifies admin roles, and updates player streak on successful match.	p_email VARCHAR(150), p_ip VARCHAR(45), p_is_success BOOLEAN	p_user_id INT (User ID), p_role VARCHAR(20), p_status VARCHAR(20), p_message VARCHAR(200)
sp_update_streak	Procedure	Updates login dates, increments consecutive streak days.	p_player_id INT	None
sp_approve_player	Procedure	Approves, rejects, suspends, or restores player registration.	p_admin_id INT, p_player_id INT, p_action ENUM('Approve', 'Reject', 'Suspend', 'Restore')	None
sp_delete_user	Procedure	Soft-deletes user record and logs administrator audit action.	p_admin_id INT, p_target_id INT	None
sp_update_profile	Procedure	Updates user account credentials.	p_user_id INT, p_first_name VARCHAR(100), p_last_name VARCHAR(100), p_phone VARCHAR(50), p_password_hash VARCHAR(255)	None
sp_request_account_deletion	Procedure	Logs deletion requests from users.	p_player_id INT	None
sp_create_password_reset_token	Procedure	Creates and logs a temporary reset token for password recover.	p_user_id INT, p_token_hash VARCHAR(255), p_expires_at DATETIME	p_token_id INT
sp_use_password_reset_token	Procedure	Consumes reset token and overwrites password hash for target user.	p_token_hash VARCHAR(255), p_new_hash VARCHAR(255)	p_message VARCHAR(100)

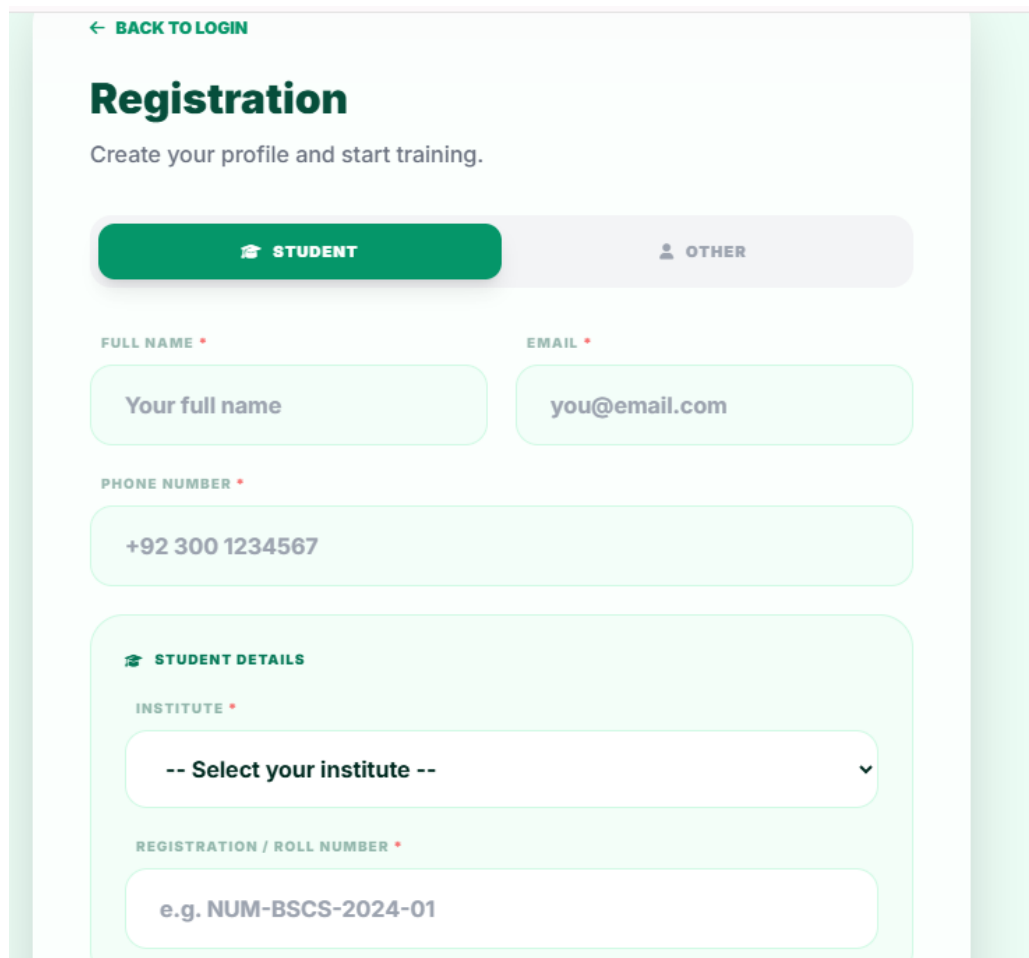
Object Name	Type	Description	Input Parameters	Output / Return Value
sp_log_simulation	Procedure	Saves a record of simulation viewing and registers completed drills.	p_player_id INT, p_simulation_id INT, p_time_spent INT, p_completed BOOLEAN	None
sp_submit_ticket	Procedure	Registers a support assistance ticket with attachments.	p_player_id INT, p_subject VARCHAR(200), p_description TEXT, p_category_id INT, p_attach_url VARCHAR(500)	p_ticket_id INT
sp_respond_ticket	Procedure	Resolves or updates ticket logs and appends admin responses.	p_admin_id INT, p_ticket_id INT, p_description TEXT, p_new_status ENUM('Open', 'In_Progress', 'Resolved', 'Closed'), p_attach_url VARCHAR(500)	None
sp_manage_faq	Procedure	Upserts database FAQ records.	p_faq_id INT, p_admin_id INT, p_question TEXT, p_answer TEXT	None
sp_manage_event	Procedure	Upserts Notice Board announcements and attachments.	p_admin_id INT, p_event_id INT, p_event_name VARCHAR(200), p_event_date DATETIME, p_venue VARCHAR(200), p_description TEXT, p_image_url VARCHAR(500), p_reg_link VARCHAR(500)	None
sp_archive_event	Procedure	Toggles archive flag for events.	p_admin_id INT, p_event_id INT	None

Object Name	Type	Description	Input Parameters	Output / Return Value
sp_manage_simulation	Procedure	Upserts simulation drills.	p_admin_id INT, p_sim_id INT, p_name VARCHAR(100), p_duration INT, p_category_id INT	None
sp_soft_delete_simulation	Procedure	Deactivates drills from catalog.	p_admin_id INT, p_sim_id INT	None
sp_manage_quote	Procedure	Upserts daily inspiration quotes.	p_admin_id INT, p_quote_id INT, p_text VARCHAR(500), p_author VARCHAR(100), p_category ENUM('Batting', 'Bowling', 'General', 'Mindset'), p_active BOOLEAN	None
sp_get_daily_quote	Procedure	Serves daily categorized quotes.	p_player_id INT	p_quote_id INT
sp_manage_pro_player	Procedure	Upserts professional profiles and career settings.	p_admin_id INT, p_player_id INT, p_name VARCHAR(100), p_nationality VARCHAR(50), p_role ENUM('Batsman', 'Bowler', 'All-Rounder', 'Wicket-Keeper'), p_style ENUM('Left Hand', 'Right Hand', 'Spin', 'Fast'), p_biography TEXT	None
sp_create_organization	Procedure	Adds new institutes to system.	p_name VARCHAR(100), p_province VARCHAR(50), p_city VARCHAR(50), p_country VARCHAR(50)	p_org_id INT
sp_update_organization	Procedure	Modifies existing institute metadata.	p_org_id INT, p_name VARCHAR(100)	None

Object Name	Type	Description	Input Parameters	Output / Return Value
sp_delete_organization	Procedure	Deletes institutes from listings.	p_org_id INT	None
sp_delete_quote	Procedure	Deletes quotes from databases.	p_quote_id INT	None
sp_delete_pro_player	Procedure	Deactivates pro player records.	p_player_id INT	None
trg_ticket_status_history	Trigger	Logs changes in Ticket_History.	None (Triggered AFTER UPDATE on TICKETS)	None
trg_admin_action_on_user_delete	Trigger	Automatically updates actions audit when user is soft-deleted.	None (Triggered AFTER UPDATE on USER)	None
trg_player_status_log	Trigger	Records edits made to player registration details.	None (Triggered AFTER UPDATE on PLAYER)	None
vw_pending_players	View	Shows players awaiting approval.	None	Relational Table
vw_all_players	View	Combines player profiles.	None	Relational Table
vw_player_dashboard	View	Aggregates user dashboard stats.	None	Relational Table
vw_admin_action_log	View	Lists system activity logging.	None	Relational Table
vw_open_tickets	View	Lists unresolved help tickets.	None	Relational Table
vw_simulation_catalog	View	Shows available cricket drills.	None	Relational Table
vw_simulation_logs	View	Displays players drill histories.	None	Relational Table
vw_admin_overview	View	Shows admin metrics and counts.	None	Relational Table
vw_pro_players	View	Connects pro players with stats.	None	Relational Table

4.4. System Interface Screenshots

This section presents the graphical user interface (GUI) layouts developed for the Namal Criko platform, demonstrating the operational layout across critical user checkpoints including authentication, player interaction, and administrative control.



The image shows a web registration form for 'Namal Criko'. At the top left, there is a link '← BACK TO LOGIN'. The main heading is 'Registration' in a large, bold, dark green font. Below it, a subtitle reads 'Create your profile and start training.' There are two tabs: 'STUDENT' (active, highlighted in green) and 'OTHER' (inactive, light grey). The form contains several input fields: 'FULL NAME' with placeholder 'Your full name', 'EMAIL' with placeholder 'you@email.com', and 'PHONE NUMBER' with placeholder '+92 300 1234567'. Below these is a section titled 'STUDENT DETAILS' which includes an 'INSTITUTE' dropdown menu with the text '-- Select your institute --' and a 'REGISTRATION / ROLL NUMBER' field with the placeholder 'e.g. NUM-BSCS-2024-01'.

← BACK TO LOGIN

Registration

Create your profile and start training.

STUDENT OTHER

FULL NAME *

EMAIL *

PHONE NUMBER *

STUDENT DETAILS

INSTITUTE *

REGISTRATION / ROLL NUMBER *

Figure 4.2: Namal Criko Registration Interface

[b]0.49

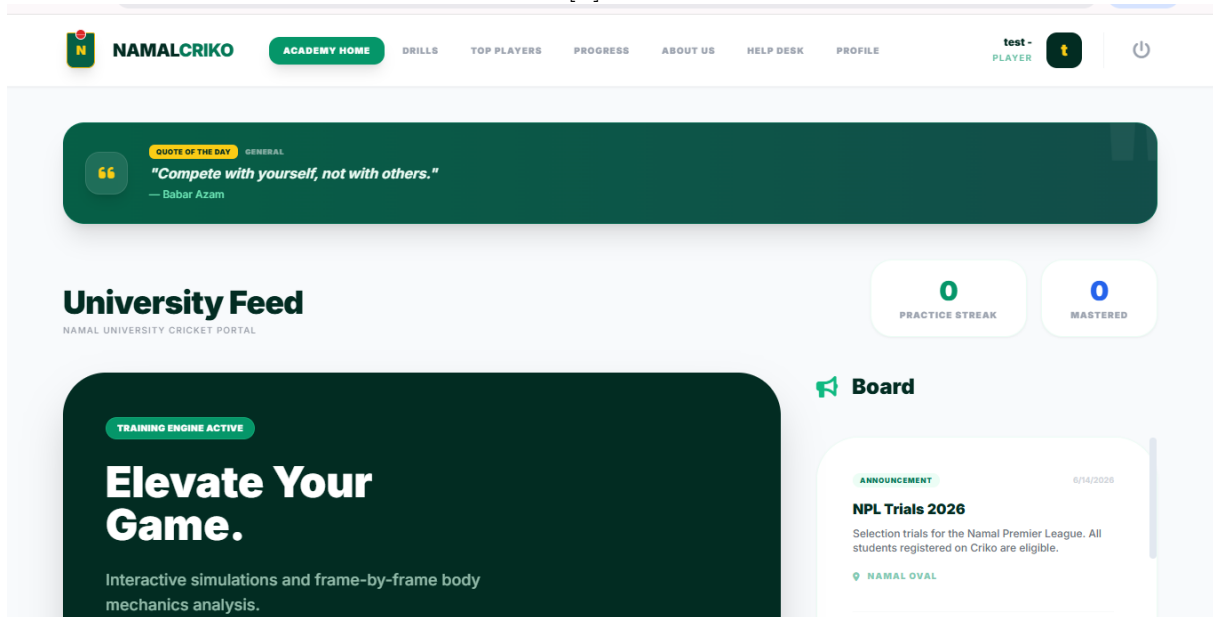


Figure 4.3: Player Dashboard Statistics View

[b]0.49

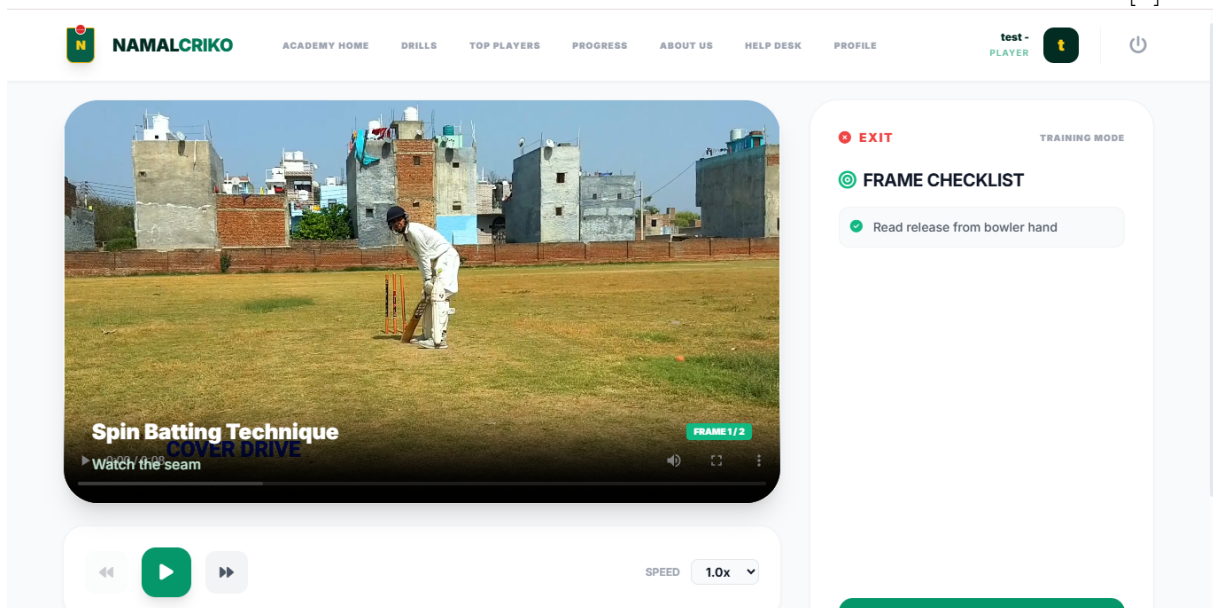
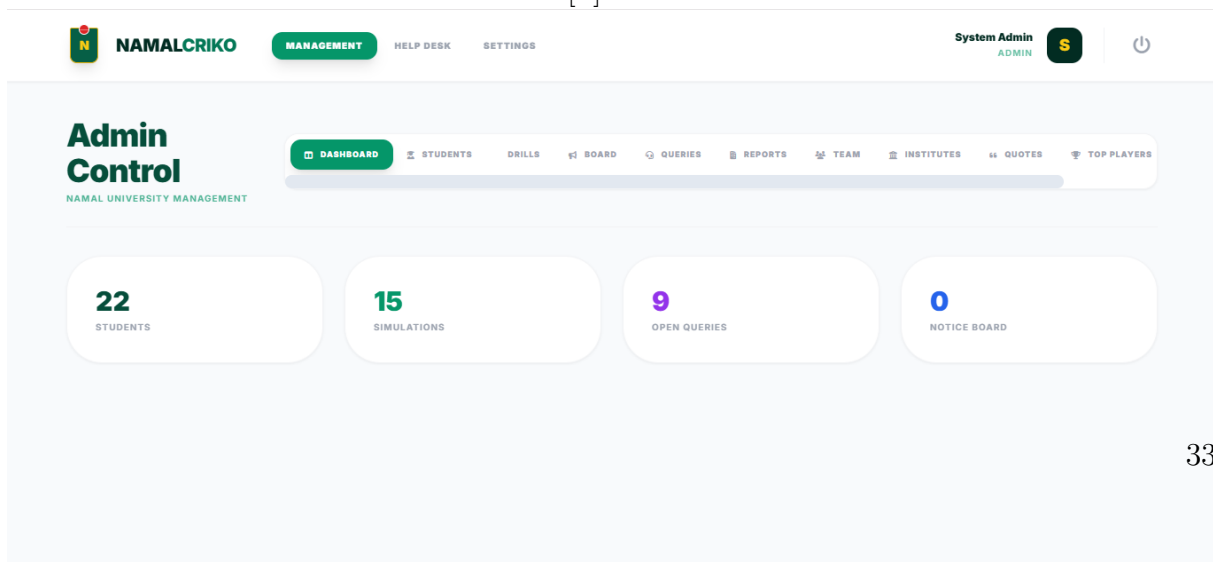


Figure 4.4: Cricket Technique Simulation Viewer

[b]0.49



References

Appendix: AI Usage and Prompts

- Help me design an ERD for a cricket simulation trainer database (MySQL) based on the provided proposal. Include entities for user management (manual admin approval, no email verification), simulation catalogue (categories, techniques, steps, key points, assets), performance tracking (logs, streaks), help desk (tickets, responses, history), Top Players (professional statistics by format), Quote of the Day (with 7-day non-repeat), events, password reset, login attempts, and admin audit logs.
- Check my ERD and help me out in finding bugs related to entity relations of missing entities.
- Give me a sample doc as per given sample along with group members and other headers in accordance with proposal doc.
- List out all entities and their attributes from attached ERD image.
- Figure out missing attributes from ERD and proposal doc.